IEICE TRANS. FUNDAMENTALS, VOL.E97–A, NO.1 JANUARY 2014

PAPER *Special Section on Cryptography and Information Security*

A Digital TRNG Based on Cross Feedback Ring Oscillators

Lijuan LI[†a)], ***Nonmember and*** **Shuguo LI**[†b)], ***Member***

SUMMARY In this paper, a new digital true random number generator based on Cross Feedback Ring Oscillators (CFRO) is proposed. The random sources of CFRO lie in delay variations (jitter), unpredictable transition behaviors as well as metastability. The CFRO is proved to be truly random by restarting from the same initial states. Compared with the so-called Fibonacci Ring Oscillator (FIRO) and Galois Ring Oscillator (GARO), the CFRO needs less than half of their time to accumulate relatively high entropy and enable extraction of one random bit. Only a simple XOR corrector is used to reduce the bias of output sequences. TRNG based on CFRO can be run continuously at a constant high speed of 150 Mbps. For higher security, the TRNG can be set in stateless mode at a cost of slower speed of 10 Mbps. The total logical resources used are relatively small and no special placement and routing is needed. The TRNG both in continuous mode and in stateless mode can pass the NIST tests and the DIEHARD tests.
key words: *digital TRNG, FPGA, stateless, metastability, chaos*

1. Introduction

Random number generators (RNGs) are essential elements in designing cryptographic protocols and algorithms. They are used to generate secret or public keys, initialize vectors, and provide padding values, nonces (numbers used once), etc. There are two main subgroups for RNGs: true random number generators (TRNGs) and pseudo random number generators (PRNGs). TRNGs exploit physical entropy sources to generate random numbers, while PRNGs are based on deterministic algorithms and their unpredictability relies on the random seeds provided by TRNGs. Though PRNGs are mostly fast and have good statistical properties, they are vulnerable to attackers if the seed and the algorithm are known. TRNGs are preferred for security considerations.

Conventional TRNGs use physical noise (such as thermal noise in resistors and/or shot noise in PN-junctions), analog voltage-controlled oscillators or chaos circuits to generate random sequences. These TRNGs usually contain analog components requiring a special custom layout. When migrated to new platforms or technologies, they need a redesign that results in an increased budget and more time-to-market. A more practical way is to implement TRNGs with digital integrated circuits only.

Digital TRNGs are mainly based on delay variations of logic gates, or analog behaviors of logic gates between two logic levels (e.g., metastability), or their combination. For the former, one common type of TRNG designs exploits randomness from jitter of free-running ring oscillators. A classic design called coupled oscillators is to sample the output of a fast oscillator on the rising edge of a slower clock using a D-type flip-flop [1]. If the sampling clock is generated by another ring oscillator, the two rings tend to couple with each other and the synchronization of oscillators is difficult. Fischer in [2] proposed a TRNG design that utilizes an on-chip PLL to generate the second clock signal from the the system clock. Since PLLs are not available in all FPGAs, we prefer designs with fundamental logic gates only.

Sunar in [3] proposed another classic TRNG design by using multiple ring oscillators. Their outputs are XORed and then sampled with a D-type flip flop. The basic idea is to populate the output waveform with transition zones. The authors provided a mathematical model based on a set of unrealistic assumptions [4]. And a huge amount of ring oscillators (110 rings with 13 inverters in [3]) are needed to guarantee the high entropy of random bits. Wold in [5] has enhanced the Sunar's design by adding an extra D-type flip-flop after each free-running RO, so that even for a reduced number of ROs, the postprocessing is not necessary. Nevertheless, Wold's method introduces pseudo randomness to the internal raw signals and this pseudo randomness is impossible to remove [6], so the design may be manipulated by attackers.

Another type of entropy source comes from the metastability of a latch or a flip-flop. The metastability is obtained by violating the setup and hold time constraints. In this case, the output of a latch (or a flip-flop) lingers or oscillates between logic levels [7], [8]. Since the metastability events are relatively rare and sensitive to temperature and voltage changes, RNGs purely based on metastability are not very reliable and may be heavily biased.

A recent design [9] used a new type of ring oscillators that works in two modes: metastability (entropy accumulation) mode and oscillation mode. In the metastability mode, the loops of individual inverters are closed (using multiplexers) to first reach a metastable state. After disconnecting the feedback loops, inverters are switched to form a single RO and start to oscillate. Metastability events provide random starting conditions for the oscillation mode. After the foregoing metastable events have resolved, signals are sampled with a delayed clock through a D-type flip flop. Without enough circuit details, it is hard to evaluate the quality of

Manuscript received March 12, 2013.
Manuscript revised July 13, 2013.
[†]The authors are with the Department of Microelectronics and Nanoelectronics/Institute of Microelectronics, Tsinghua University, Beijing, 100084, P.R. China.
a) E-mail: li-lj11@mails.tsinghua.edu.cn
b) E-mail: lisg@tsinghua.edu.cn
DOI: 10.1587/transfun.E97.A.284

Copyright © 2014 The Institute of Electronics, Information and Communication Engineers

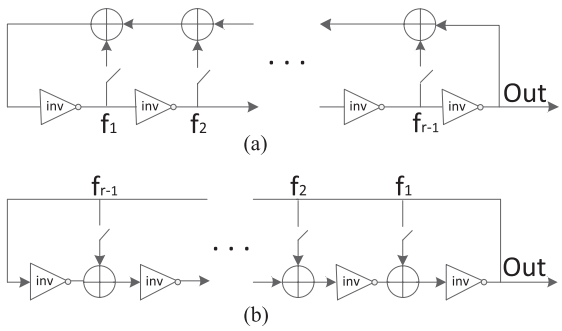

Fig. 1 (a) Fibonacci ring oscillator (b) Galois ring oscillator.

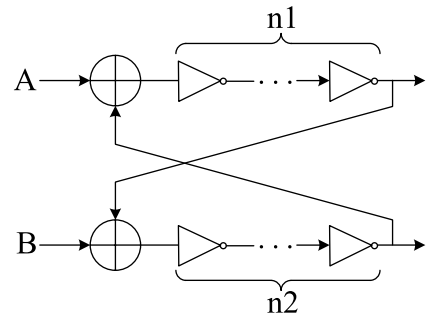

Fig. 2 Cross feedback loop.

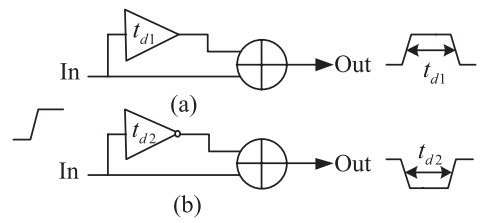

Fig. 3 (a) '1' pulse generator (b) '0' pulse generator.

the design. However, the generator requires stable temperature and voltage to function well, which may imply that this TRNG design is not reliable.

In [10], Golić proposed a different TRNG design borrowing concepts from linear feedback shift registers (LFSR). By replacing shift registers with inverters, he obtained new types of ring oscillators - Fibonacci Ring Oscillators (FIRO) and Galois Ring Oscillators (GARO) shown in Fig. 1. The polynomials used in these oscillators need to meet some requirements to avoid a fixed point. The output signals take on both pseudo randomness and true randomness. The original generator uses powerful post-processor (an irregularly clocked 64-bit LFSR) to reduce the bias of the output sequences.

In [4], Dichtl and Golić enhanced the original principle by restarting the oscillators from identical initial conditions. They implemented these oscillators on a Xilinx Spartan-3 Starter Kit board and managed to generate random bits at a speed of 7.14 Mbps (6.25 Mbps) in restart mode. A novel sampling method that extracts two random bits at a time is also proposed and thus the entropy rate can be almost doubled. The enhanced design is inner testable and the entropy included in the output signals was shown to be very high.

Enlightened by [4], [10], we propose a digital TRNG based on a new spontaneous oscillating circuit - Cross Feedback Ring Oscillator (CFRO). The rest of this paper is organized as follows: The architecture of CFRO is present in Sect. 2 with a detailed description of the automatic oscillating process. In Sect. 3, the random source of CFRO is analyzed and a restart test is carried out to prove true randomness of CFRO. A performance comparison with FIRO and GARO is present as well on standard deviations evolution of output voltages. The circuit details are discussed in Sect. 4 and the postprocessing method is described in Sect. 5. In Sect. 6, the stateless mode of CFRO is introduced. A test strategy for statistical evaluation and the test results are presented in Sect. 7. Conclusions are given in Sect. 8.

2. A New Spontaneous Oscillating Circuit

The spontaneous oscillating circuit we propose mainly includes two parts: cross feedback loops and pulse generating circuits. The cross feedback loop is shown in Fig. 2. It contains two exclusive-OR (XOR) gates and two inverter chains each including n1 and n2 inverters. The loops can be divided into two types according to the parity of the total number of inverters. One type contains an even number of inverters (both n1 and n2 are even or odd). The XOR gate acts as an inverter or a buffer when one input is fixed to '1' or '0' respectively. In other words, the loop amounts to a ring with an even number of inverters when A = B = '0' or A = B = '1', or a ring with a odd number of inverters when A = '1', B = '0' or A = '0', B = '1'. Therefore, the loop settles down to a stable state in the former case while oscillating asynchronously in the latter case. Another type of cross feedback loops contains a odd of number of inverters (either n1 or n2 is odd, the other is even).In this case, the loop oscillates when A = B = '0' or A = B = '1' and stabilizes when A and B are different.

Another basic part in our design is pulse generating circuits. Figure 3(a) shows a '1' pulse generator. In static state, the two inputs of the XOR gate are the same so that the output is '0'. The transition of the input makes the two inputs of the XOR gate different temporarily, thus the output turns to '1'. After a delay of t_{d1}, the disturbance is removed and the output remains '0' again. As a result, a '1' pulse of width t_{d1} is generated. Similarly, the '0' generator in Fig. 3(b) can produce a '0' pulse of width t_{d2}, when the transition of the input comes.

By connecting several cross feedback loops with pulse generators, we can get a new oscillating circuit - Cross Feedback Ring Oscillators (CFRO). Various combinations can be workable if only the circuit won't get stuck. For example, Fig. 4 shows a typical CFRO with a cascade of three cross feedback loops. To guarantee that the circuit can oscillate automatically after restart, inputs to node A and node B should be different in static. Therefore, we use a '1' pulse generator and a '0' pulse generator, each including two dif-

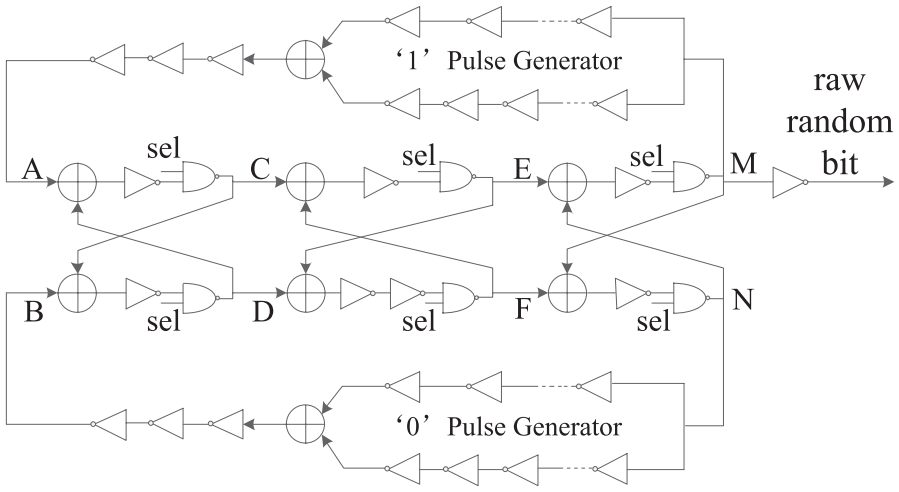

Fig. 4 Cross feedback ring oscillator.

ferent delay paths. The inverter numbers of two delay paths are both even or odd for the '1' pulse generator, while for the '0' pulse generator, one is even, and the other is odd. The pulse generators take outputs of the last cross feedback loop as inputs and give feedback to the first cross feedback loop via several inverters. The two output inverters of each cross feedback loop are replaced by NAND gates with a select signal (sel), so that the CFRO can stop oscillating when sel = '0'.

The oscillation mode of CFRO can be described as follows. After restart, A = '1', B = '0', the first loop starts to oscillate, and the rest of the loops oscillate along with the first one, causing the voltage transition of node M and node N. Through two pulse generators with different delays, the feedback signals to node A and node B may be '1' or '0' or even metastable. As a result, the first loop may oscillate as before or change phases to continue oscillating or turn to a steady state temporarily. In reality, the minimum interval between feedback signals is only delay of an inverter. The feedback is so fast that the circuit would not stay in some state for a long time. Usually, before the voltage of some node translates to a full logic high level or a full logic low level, another feedback pulse has come, forcing the state of the node to change again. By simulating in Hspice and observing with the Agilent oscilloscope, we find that the CFRO oscillates in an analog level, somewhat like chaotic and random. We will further discuss the randomness of the CFRO in the next section.

3. Randomness Tests

If we consider logic gates as ideal function components with constant delays, the CFRO can be viewed as a binary iterative system with unequal time-delayed feedback paths. This iterative system behaves like a chaotic system, which is deterministic but sensitive to initial states and intermediate variables. However, this assumption is unrealistic because logic gates are not that ideal. In fact, every logic gate has transition delay as well as propagation delay. These delays can be influenced easily by various internal and external noise of circuits, and thus fluctuate with a small random variation (jitter). What's more, if a pulse arrives when the gate is in transition, the output behavior would be very complicated and unpredictable [11], [12]. The output becomes totally chaotic and analog during a continuous feedback process. The metastability events are easier to occur and accumulate through the iteration process. Therefore, the random sources of the CFRO can be summarized to delay variations (jitter), unpredictable transition behaviors as well as metastability, which make the oscillation of the CFRO random, chaotic, analog as well as metastable. To prove the oscillation of the CFRO is truly random but not deterministic, we adopt the restart approach and standard deviation analysis in [4].

3.1 Restart Test

We implement the 3-stage CFRO described in Fig. 4 on the Altera Cyclone IV E FPGA board, and observe the oscillogram with the Agilent oscilloscope. The '1' pulse generator contains 5 inverters and 9 inverters each in two paths, while the '0' pulse generator contains 8 inverters and 11 inverters each in two paths. The CFRO is restarted from the same initial conditions with a long enough interval for restoration. The results are shown in Fig. 5. In this figure, the horizontal axis is the time, and the period of each run is 100 ns. The vertical axis is the output voltage of raw random sequences. Only 20 curves are given due to limited space. From Fig. 5 we can see that these curves are only similar in the beginning because of the identical starting conditions, and soon after they diverge from each other very quickly, which implies a rapid accumulation of entropy. We can also see that the CFRO oscillates in an analog level.

3.2 Standard Deviation Analysis

According to [4], the randomness of a TRNG can be eval-

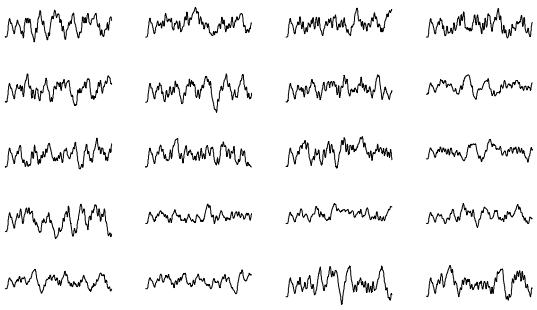

Fig. 5 Output voltage of raw random sequences, each 100 ns long, of a 3-stage CFRO.

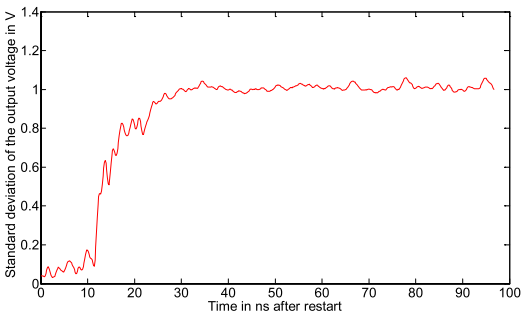

Fig. 6 Standard deviation of the output voltage of 1000 restarts of a 3-stage CFRO.

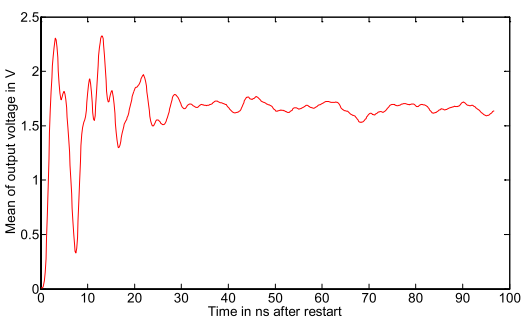

Fig. 7 Mean value of the output voltage of 1000 restarts of a 3-stage CFRO.

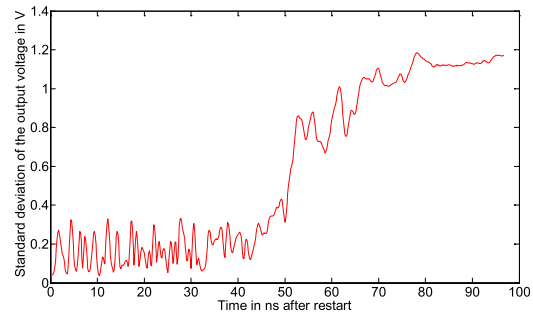

Fig. 8 Standard deviation of the output voltage of 1000 restarts of a FIRO, with feedback polynomial $1 + x^2 + x^4 + x^5 + x^{12} + x^{13} + x^{15} + x^{16} + x^{18} + x^{20}$.

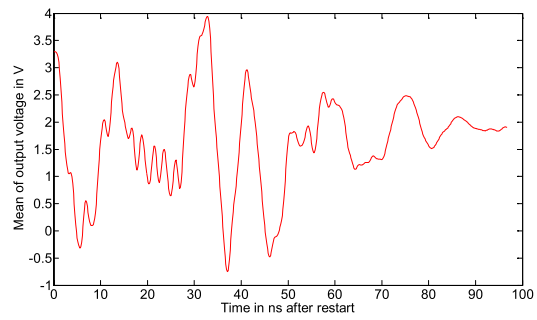

Fig. 9 Mean value of the output voltage of 1000 restarts of a FIRO, with feedback polynomial $1 + x^2 + x^4 + x^5 + x^{12} + x^{13} + x^{15} + x^{16} + x^{18} + x^{20}$.

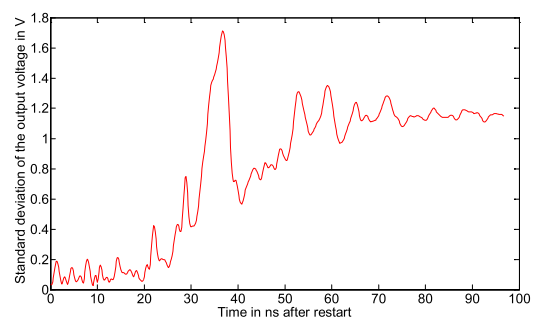

Fig. 10 Standard deviation of the output voltage of 1000 restarts of a GARO, with feedback polynomial $1 + x^2 + x^3 + x^7 + x^{16} + x^{17} + x^{19} + x^{21}$.

uated by the standard deviation of the output voltage over time. When the standard deviation is relatively large, it indicates that the output has accumulated enough entropy. Then it is easy and reliable to extract one random bit. For a relatively small standard deviation, extracting in such case will result in a heavily biased random sequence and this bias will strongly depend on the circuit implementation. From the oscillograms with the oscilloscope, we can observe that it takes about 30–50 ns for CFRO to return to initial states. Hence, the CFRO is restarted with an interval of 100 ns for restoration. The run time of each restart is 100 ns. The mean value and the standard deviation of the output voltage for 1000 randomly chosen restart curves are shown in Fig. 6 and Fig. 7 respectively.

As we can see from Fig. 6 and Fig. 7, the standard deviation grows fast in 10–20 ns. After 25–30 ns, the standard deviation becomes significantly large and both the mean value and the standard deviation achieve relatively stable values. The entropy extraction since then is robust and reliable. Therefore, the CFRO can produce random bits with high entropy and the accumulation of entropy is very fast.

For comparison, we also implement FIRO and GARO [4] on the same FPGA board. The feedback polynomials for the best design are $1 + x^2 + x^4 + x^5 + x^{12} + x^{13} + x^{15} + x^{16} + x^{18} + x^{20}$ with five inverters per inverter unit for the FIRO and $1 + x^2 + x^3 + x^7 + x^{16} + x^{17} + x^{19} + x^{21}$ with three inverters per inverter unit for the GARO. Restarting the FIRO and the GARO under the same conditions with the CFRO, the results are displayed in Fig. 8–Fig. 11.

Different from the conclusions in [4], it takes about 70 ns for the FIRO to achieve relatively high standard de-

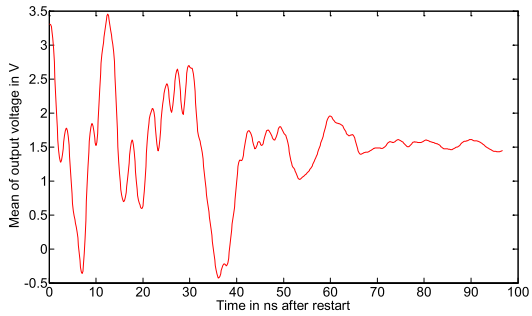

Fig. 11 Mean value of the output voltage of 1000 restarts of a GARO, with feedback polynomial $1 + x^2 + x^3 + x^7 + x^{16} + x^{17} + x^{19} + x^{21}$.

viation. The mean value has not yet stabilized. As for the GARO, the standard deviation reaches high values after about 60 ns. After 65–70 ns, both the mean value and the standard deviation reach relatively stable values. The difference with the results in [4] is due to different FPGAs used. But implemented on the identical FPGA, the CFRO we propose really outperforms the FIRO and the GARO. In fact, each time after restart, the output voltages of both the FIRO and the GARO still keep surprisingly similar in the previous 40 ns, while the CFRO has already been able to produce one reliable random bit after 30 ns.

The random sources of CFRO, FIRO, GARO are similar but their feedback mechanisms are different. For the FIRO (Fig. 1(a)), all selected internal nodes including the output node are XORed and input to the header of the ring. For the GARO (Fig. 1(b)), each selected internal node is XORed with the output node and then input to the subsequent inverter. The feedback sources in both FIRO and GARO are relatively simple so that long primitive polynomials have to be used to increase complexity and randomness. In reality, those nodes are highly relevant to each other because they come from one inverter chain. Unlike the FIRO and the GARO, our proposed CFRO is based on the neighborhood interaction (cross feedback between two inverter chains). The feedback paths are shorter and the feedback signals are more independent of each other. What's more, the pulse generators with longer delay paths can absorb more intrinsic noises in the circuit. Therefore, the CFRO can accumulate entropy faster and better with less logical resources.

4. Circuit Details

To implement CFRO on FPGA board, several varieties should be taken into account. The type of the first cross feedback loop and the types of pulse generators should be matched to guarantee spontaneous oscillation. For cross feedback loops, short inverter chains are preferred thus the loops will produce signals with smaller amplitude, which will provoke metastability more often. The inverters cannot be removed because the loops will stop oscillating in such case. Accordingly, we choose loops with two inverters in each path and loops with two inverters and three inverters each in two paths. As shown in Fig. 4, the adjacent loops are of different types. In fact, we have also implemented the CFRO including three identical loops with two inverters in each path, which is workable too.

Each pulse generator has two delay paths. The delay difference between the two paths decides the pulse width, which directly influences the duration of tail-to-head feedback (i.e., from M to A, from N to B). A delay difference of 3-5 inverters is suggested by experiments, which is less than the period of the cross feedback loop. Because one of the random sources of CFRO comes from delay variations (jitter), relatively long delay paths with 5-15 inverters are chosen to absorb more intrinsic noises in oscillating. In fact, designs with different configurations have been evaluated with the NIST test and Diehard test later discussed in Sect. 7. One available design is '1' pulse generator with 5 inverters and 9 inverters each in two paths and a '0' pulse generator with 8 inverters and 11 inverters each in two paths.

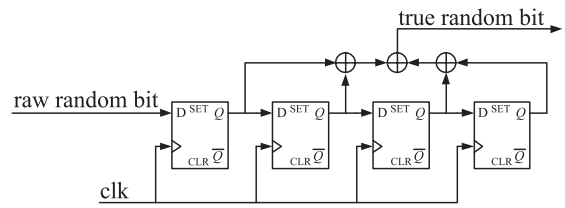

Fig. 12 A 4-stage XOR corrector.

5. Bias Correction

Good TRNGs are supposed to have a uniform distribution of '0' and '1' at their output, namely the probabilities for '0' and '1' should be 0.5. We compute the bias as

$$bias = |E[b(n)] - 0.5| \approx \left|\frac{N_1}{N} - 0.5\right|$$

where N_1 is the number of $b(n) = 1$ for $n = 0, 1...N - 1$. The record length of N is 100 000 000 bits. The bias should converge to 0 if the tested sequence is truly random.

We run the CFRO continuously and sample the output with a D-type flip flop. When the frequency of the sampling clock is 50 MHz, 100 MHz and 150 MHz, the raw random sequences have a bias of 0.02085, 0,02338, 0.01473 respectively. To reduce the bias of the output bits, we use a 4-stage XOR corrector shown in Fig. 12. The bias of the output bits is decreased only if the original bits are independent. The main advantages of the XOR corrector are its simplicity and the ability to maintain a constant output bit rate [13]. The corrected bias is as small as 0.0002890, 0.0000835 and 0.0000005 at a sampling frequency of 50 MHz, 100 MHz and 150 MHz respectively.

6. Stateless Mode

In [14], Bucci and Luzzi introduced the concept of stateless generators. The aim of stateless generators is to remove the dependence of the current state on its previous state. The

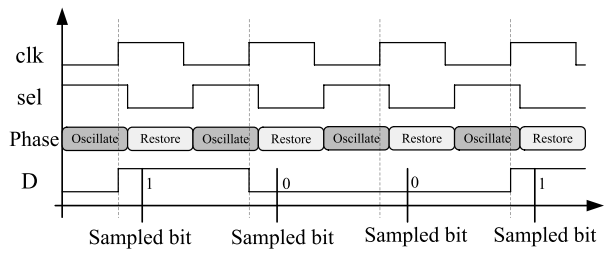

Fig. 13 The timing diagram of the stateless CFRO.

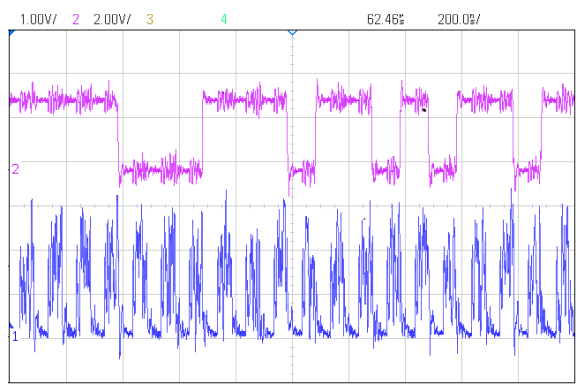

Fig. 14 The sampled bit and the raw output of a stateless CFRO, with a select signal, 50 ns for oscillating and 50 ns for restoring.

stateless hypothesis can be fulfilled by resetting TRNG to a constant value for every state variable in both the entropy source and the post-processor, before the generation of a new random bit.

Our proposed CFRO can be set to identical initial conditions when sel = '0'. From conclusions in Sect. 3.2, the CFRO can get a reliable random bit in 25–30 ns after restart, with a waiting time about 30–50 ns for returning to initial conditions. Therefore, we use a select signal with a period of 100 ns, and both the running time and the waiting time are 50 ns. The timing diagram of the stateless CFRO is shown in Fig. 13. The select signal is a delayed inverse of the clock signal, thus when the rising edge of clock comes, the output is sampled during the oscillation phase but not the restoration phase. We observed the stateless CFRO with the Agilent oscilloscope. The raw output (the blue one in Fig. 14) obviously has oscillation phase and restoration phase. The sampled bit (the red one in Fig. 14) may be '0' or '1' (random). The sampled sequence has a bias of 0.0157 with a record length of 100 000 000 bits. After postprocessing with a 4-stage XOR corrector, the bias is reduced to 0.000158.

7. Statistictal Evaluation

In order to evaluate the output quality of our TRNG based on CFRO, we use both NIST SP 800-22 [15] and DIEHARD [16] statistical tests. From the experiment results, the NIST tests are more picky and time-consuming than the DIEHARD tests. Designs which pass the NIST tests usually can pass the DIEHARD tests, but not vice verse. For the NIST tests, it is recommended that the Frequency test be run first, since this supplies the most basic evidence for the existence of non-randomness in a sequence, specifically, non-uniformity. If this test fails, the probability of failing is high for other tests. The Runs test is the most difficult statistical test of NIST for our design to pass, while the Linear Complexity test is the most time-consuming one. Therefore, our testing strategy is to run six types of NIST tests first (Frequency test, Block Frequency test, Runs test, Longest Run test, Rank test and FFT test), which costs much less time and basically indicates a good random number generator. Only when the six tests are passed, the rest of NIST tests and DIEHARD tests are performed for a more precise evaluation.

We have tested the CFRO both in continuous mode (50 MHz, 100 MHz, 150 MHz) and stateless mode (10 MHz). Designs without postprocessing failed tests mainly due to the frequency tests (even a design with a bias of 0.001 still fails the tests). With a 4-stage XOR corrector, the TRNG in both of the two working modes can pass the NIST tests and the DIEHARD tests.

Table 1 shows the NIST testing results of our proposed TRNG in stateless mode. The significance level α is 0.01 and the tested random sequence consists of 1000 subsequences of 1 000 000 bits. Each subsequence generates a p-value for every test. This value is the probability that a true random number generator produces a sequence less random than the tested RNG with regards to a certain property. A p-value equals to 1 means that the sequence is perfectly random and 0 that it is not at all. The distribution of p-values is examined to ensure uniformity. The interval between 0 and 1 is divided into 10 sub-intervals (C1-C10), and the p-values that lie within each sub-interval are counted and displayed in Column 1- Column 10. The uniformity is also examined with a χ^2 test [15] in Column 11. If p-value$_T \geq 0.0001$, then the subsequences are considered uniformly distributed. Column 12 shows the proportion of sequences that pass (i.e., proportion of p-values greater than α). The proportion must belong to the following interval for the sequence to be considered random:

$$\hat{p} \pm 3\sqrt{\frac{\hat{p}(1-\hat{p})}{m}}$$

where $\hat{p} = 1 - \alpha$ and m is the subsequence number. For our test, the confidence interval is $0.99 \pm 3\sqrt{\frac{0.99\times0.01}{1000}} = 0.99 \pm 0.0094392$ (i.e., the proportion should lie above 0.9805607). Therefore, the minimum pass rate for each statistical test is approximately 980/1000, with the exception of the random excursion (variant) test, and 607/621, for the random excursion (variant) test. Table 1 shows that both the p-values and proportions have reached the requirements for our proposed TRNG.

8. Conclusions

A new spontaneous oscillating circuit, CFRO (Cross Feed-

Table 1 NIST results of our proposed TRNG in stateless mode at 10 MHz.

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-value	Proportion	Statistical test
106	106	100	105	93	68	109	98	111	104	0.125200	990/1000	Frequency
119	109	124	107	87	97	97	82	87	91	0.032061	987/1000	Block-Freq
106	114	99	102	100	109	90	91	86	103	0.632955	991/1000	CuSum
116	96	82	100	131	105	94	91	92	93	0.033807	986/1000	CuSum
105	91	90	101	98	99	100	107	111	98	0.920383	989/1000	Runs
101	87	109	82	115	105	92	94	117	98	0.203351	993/1000	LongestRun
102	108	110	93	86	98	111	96	93	103	0.727851	990/1000	Rank
97	97	106	101	114	116	106	81	88	94	0.286836	988/1000	FFT
98	92	96	103	116	95	96	91	114	99	0.670396	991/1000	Universal
86	101	108	105	110	101	95	102	97	95	0.875539	992/1000	ApproximateEntropy
63	54	56	60	69	62	57	56	74	70	0.655585	616/621	RandomExcursions
54	58	57	70	66	70	61	54	64	67	0.785664	619/621	RandomExcursions-V
93	95	103	101	111	96	87	109	110	95	0.743915	989/1000	Serial
88	118	116	88	103	93	84	91	111	108	0.106246	994/1000	Serial
99	102	85	104	101	106	106	84	108	105	0.674543	989/1000	LinearComplexity

back Ring Oscillator) is proposed. The CFRO mainly consists of several cross feedback loops and two pulse generators. Experimental results show that the CFRO oscillates in an analog level, which is chaotic and unpredictable. The output curves of the CFRO are different each time after restart from the same initial conditions. Therefore the CFRO is truly random but not deterministic. In order to compare the quality of CFRO with FIRO and GARO, we implement them on the same FPGA board and measure the mean values and the standard deviations of the output voltages. The results show that our CFRO needs less than half of the time of FIRO and GARO to accumulate relatively high entropy and enable extraction of one reliable random bit. This is due to shorter feedback paths and a different oscillation mode. With a simple XOR corrector for postprocessing, the TRNG based on CFRO can produce nearly unbiased random bits at a constant speed. The TRNG can be run continuously at a high speed of 150 Mbps. For high-security applications, the TRNG can be restarted regularly and thus a stateless working mode is achieved at a cost of slower speed (10 Mbps).

Our design can be easily implemented on FPGA with logic only without special placement and routing. The total costs of resources are around 50 inverters (some replaced by NAND gates) and several XOR logic gates. The design both in continuous mode and stateless mode can pass the NIST tests and the DIEHARD tests.

Acknowledgments

This work was partly supported by the National High-Technology Research and Development Program of China (No. 2012AA012402), the National Natural Science foundation of China (No. 61073173), and Independent Research and Development Program of Tsinghua University (No. 2011Z05116). The author would also like to thank the editor and reviewers for their contributive comments.

References

[1] R. Fairfield, R. Mortenson, and K. Coulthart, "An LSI random number generators(RNG)," Advances in Cryptology: Proc. CRYPTO 84, LNCS 196, pp.203–230, Springer, 1985.

[2] V. Fischer and M. Drutarovský, "True random number generator embedded in reconfigurable hardware," Proc. Cryptographic Hardware and Embedded Systems - CHES 2002, LNCS 2523, pp.415–430, Springer, 2002.

[3] B. Sunar, W.J. Martin, and D.R. Stinson, "A provably secure true random number generator with built-in tolerance to active attacks," IEEE Trans. Comput., vol.56, no.1, pp.109–119, 2007.

[4] M. Dichtl and J.D. Golić, "High-speed true random number generation with logic gates only," Proc. Cryptographic Hardware and Embedded Systems CHES 2007, LNCS 4727, pp.45–62, Springer, 2007.

[5] K. Wold and C.H. Tan, "Analysis and enhancement of random number generator in FPGA based on oscillator rings," Int. J. Reconfigurable Computing, vol.2009, pp.1–8, 2009.

[6] N. Bochard, F. Bernard, and V. Fischer, "Observing the randomness in RO-based TRNG," Proc. International Conference on Reconfigurable Computing and FPGAs, pp.237–242, 2009.

[7] L. Kleeman and A. Cantoni, "Metastable behavior in digital systems," IEEE Des. Test Comput., vol.4, no.6, pp.4–19, 1987.

[8] H. Hata and S. Ichikawa, "FPGA implementation of metastability-based true random number generator," IEICE Trans. Inf. & Syst., vol.E95-D, no.2, pp.426–436, Feb. 2012.

[9] I. Vasyltsov, E. Hambardzumyan, Y.S. Kim, and B. Karpinskyy, "Fast digital TRNG based on metastable ring oscillator," CHES 2008. LNCS, vol.5154, pp.164–180, Springer, Heidelberg, 2008.

[10] J.D. Golić, "New methods for digital generation and postprocessing of random data," IEEE Trans. Comput., vol.55, no.10, pp.1217–1229, 2006.

[11] T. Kacprzak, "Analysis of oscillatory metastable operation of an R-S flip-flop," IEEE J. Solid-State Circuits, vol.23, no.1, pp.260–266, 1988.

[12] L. Reyneri, D. Corso, and B. Sacco, "Oscillatory metastability in homogenous and inhomogeneous flip-flops," IEEE J. Solid-State Circuits, vol.25, no.1, pp.254–264, 1990.

[13] R.B. Davies, "Exclusive OR (XOR) and hardware random number generators," Online. Available at: http://webnz.com/robert/, 2002.

[14] M. Bucci and R. Luzzi, "Design of testable random bit generators," CHES 2005. LNCS, vol.3659, pp.147–156, Springer, Heidelberg, 2005.

[15] NIST Special Publication 800C22. A Statistical Test Suite for Random and Pseudorandom Numbers, 2000.

[16] G. Marsaglia. DIEHARD: A Battery of Tests of Randomness, http://stat.fsu.edu/pub/diehard/, 1996.

Lijuan Li received her B.E. degree in 2011 from the School of Astronautics of Harbin Institute of Technology. Now she is a Phd Candidate with Department of Microelectronics and Nanoelectronics/Institute of Microelectronics of Tsinghua University.

Shuguo Li is a professor at the Institute of Microelectronics at Tsinghua University. His current research interests include the algorithm, design and VLSI implementation for cryptography.